Morse Code Converter

Deep Dive

Explainer document for the owner

Contents

1 What this project is

This is a small Python project with three parts:

- `morse_converter.py` — pure conversion logic: a lookup table mapping every supported character to its International Morse Code pattern, and two functions that convert text to Morse and Morse back to text. It has no user-interface code at all.
- `gui.py` — a desktop window (built with **Tkinter**, the graphical toolkit that ships with standard Python) that wraps those two functions in a live, two-way interface: you type in one box and the conversion appears in another box as you type, plus a playback panel that visually "plays back" the dots and dashes of the current Morse code *and*, new in this version, plays them as real audio tones in sync.
- `morse_audio.py` — new module that generates the audio tones and figures out, at runtime, how to actually play a sound on the machine the app happens to be running on. It has no Tkinter dependency either; `gui.py` just calls into it.

Glossary term — standard library / stdlib: the modules that ship with Python itself, needing no extra installation (`tkinter`, `wave`, `math`, `struct`, `subprocess`, `shutil`, `tempfile`, `atexit` are all stdlib). This project uses only stdlib modules, no third-party packages at all, including for the new audio feature.

Glossary term — Morse code: a way of encoding text as sequences of short and long signals ("dots" and "dashes"), historically tapped out on a telegraph key or flashed as light. Each letter, digit, or punctuation mark has its own fixed sequence, defined by international convention (this project uses the standard ITU alphabet).

Glossary term — dictionary (Python): a data structure that stores key-value pairs and looks up a value instantly given its key, similar to a paper dictionary mapping a word to its definition. This project uses one dictionary to go from character to Morse pattern, and a second, automatically built by reversing the first, to go the other way.

2 The conversion logic (`morse_converter.py`)

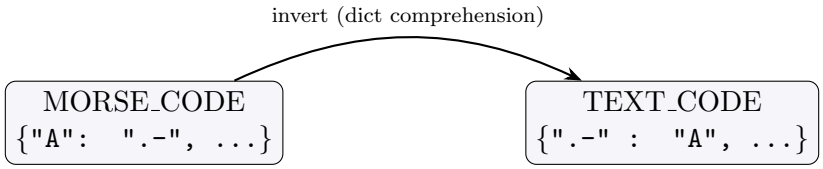
2.1 The lookup tables

The heart of the file is a single dictionary, `MORSE_CODE`, with one entry per supported symbol: the 26 uppercase letters, the 10 digits, the space character (mapped to `"/"`, explained below), and a set of punctuation marks (`! ? . , ' " / ( ) & : ; = + - _ $ @` and two accented question/exclamation marks, `¿` and `¡`). Every value is a string made only of the characters `.` (dot) and `-` (dash).

A second dictionary, `TEXT_CODE`, is built once, at import time, by inverting `MORSE_CODE`:

```
TEXT_CODE = {value: key for key, value in MORSE_CODE.items()}
```

This is a Python *dict comprehension*: a compact way of writing a loop that builds a new dictionary. Reading it as an ordinary loop, it means: "for every (character, pattern) pair already in `MORSE_CODE`, add a new entry to `TEXT_CODE` keyed by the pattern, pointing back at the character." Because every Morse pattern in the table is unique, this inversion loses no information: given any valid pattern, `TEXT_CODE` recovers exactly one character.

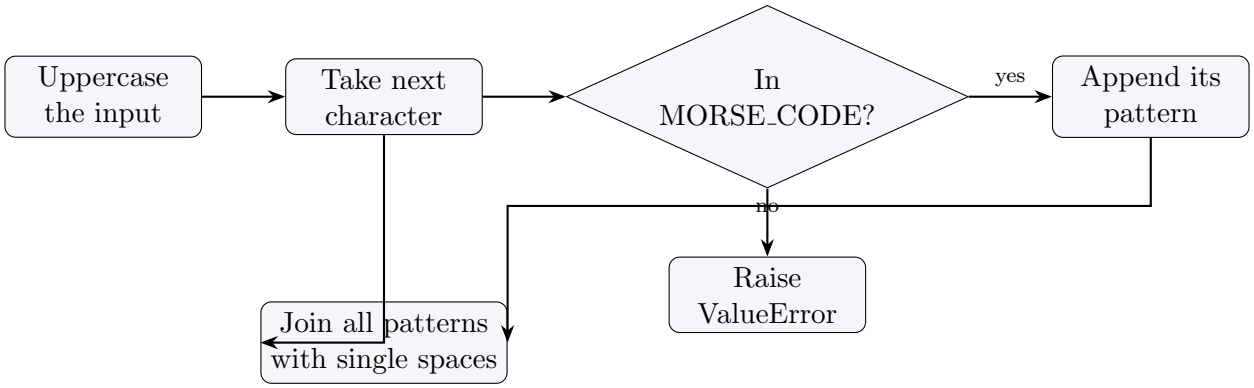


2.2 `text_to_morse(text)`: text → Morse

```
def text_to_morse(text: str) -> str:
    codes = []
    for char in text.upper():
        if char not in MORSE_CODE:
            raise ValueError(f"No Morse mapping for character: {char!r}")
        codes.append(MORSE_CODE[char])
    return " ".join(codes)
```

Step by step:

1. **Normalize case.** `text.upper()` converts the whole input to uppercase first, so `"sos"` and `"SOS"` produce the identical result — the table only has uppercase letter keys.
2. **Look up each character.** The function walks the string one character at a time. If a character has no entry in `MORSE_CODE` (an emoji, an unsupported accented letter, etc.), it deliberately raises a `ValueError` — a built-in Python exception used to signal "this input is invalid" — naming the exact offending character, rather than letting the lookup fail with a raw, less helpful `KeyError` (Python's default error when a dictionary key is missing).
3. **Join with spaces.** Each character's Morse pattern is collected into a list, then joined with single spaces. The space character itself maps to `"/"`, so a literal space in the input becomes the three characters `" / "` in the output once joined (a space before the `/`, the `/` itself, and the space after it from the join). This is the conventional Morse word separator.



2.3 `morse_to_text(morse)`: Morse → text

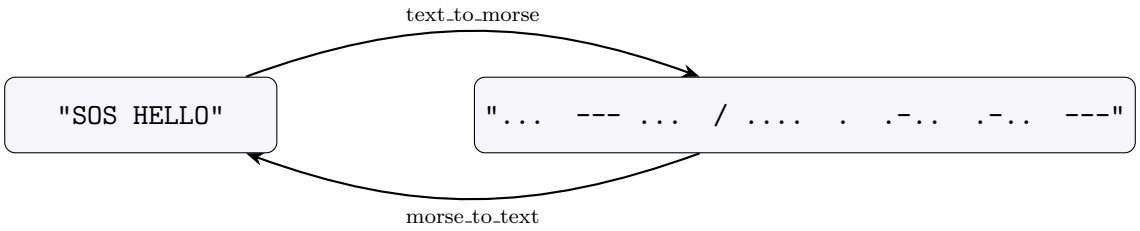
```
def morse_to_text(morse: str) -> str:
    words = morse.strip().split(" / ")
    decoded_words = []
    for word in words:
        letters = []
        for symbol in word.split():
```

```
            if symbol not in TEXT_CODE:
                raise ValueError(f"No text mapping for Morse symbol: {symbol!r}")
            letters.append(TEXT_CODE[symbol])
        decoded_words.append("".join(letters))
    return " ".join(decoded_words)
```

This function reverses `text_to_morse` in two nested passes:

1. **Split into words.** `morse.strip()` removes leading and trailing whitespace, then `.split(" / ")` cuts the string on the exact three-character sequence `" / "` (space, slash, space). This exact-string split matters: it is precisely what `text_to_morse` produces at a word boundary (see §2.2), so splitting on it, rather than on a bare `"/"`, avoids leaving stray leading/trailing spaces attached to the words on either side.
2. **Split each word into letters.** Within a word, `.split()` with no argument splits on any run of whitespace, giving one Morse pattern per letter (e.g. `"....  .  .-..  .-..  ---"` becomes five patterns).
3. **Look up and rebuild.** Each pattern is looked up in `TEXT_CODE`; an unmapped pattern raises `ValueError` naming the exact symbol, mirroring the error handling in `text_to_morse`. The recovered letters are concatenated with no separator (`"".join(letters)`) to rebuild the word, and the recovered words are joined back together with ordinary single spaces.

2.4 Round trip



Because `text_to_morse` always uppercases its input, `morse_to_text(text_to_morse(s)) == s.upper()`, not necessarily `s` itself if `s` contained lowercase letters.

2.5 Running the file directly

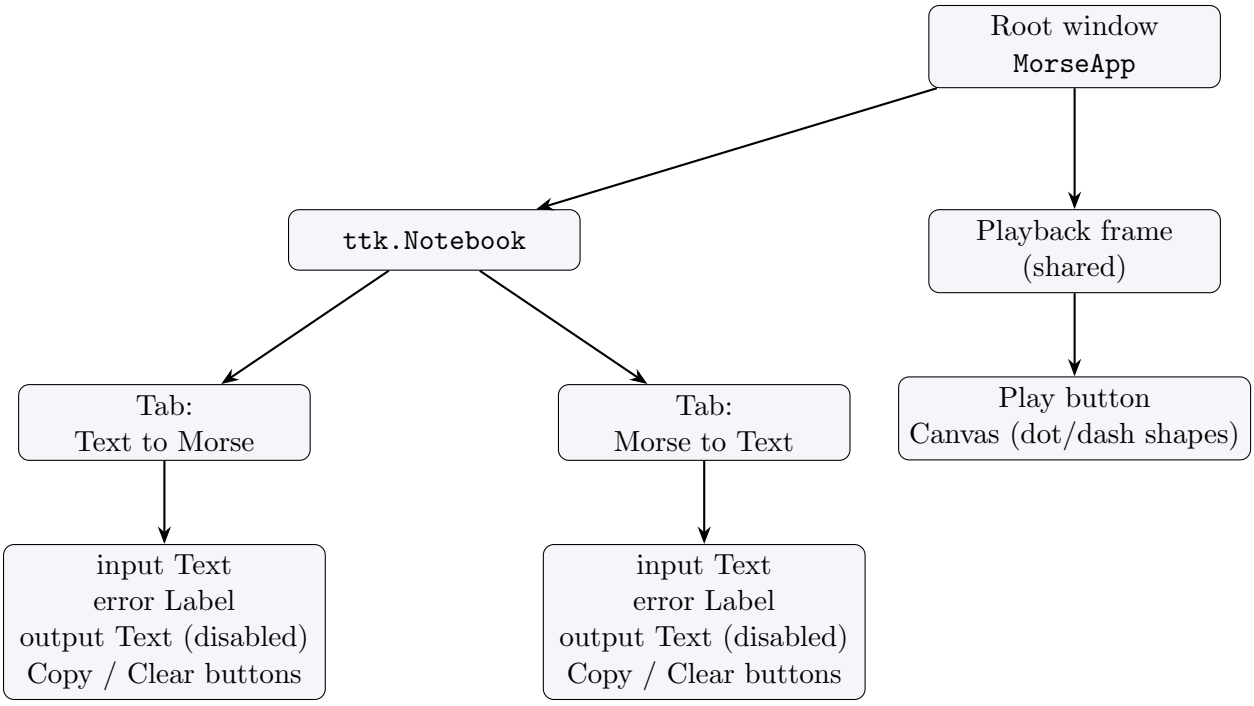
The file's `if __name__ == "__main__":` block — Python's convention for "only run this when the file is executed directly, not when it is imported by another file" — still contains the original one-line command-line prompt: it asks for a message with `input()` and prints the Morse translation. This still works exactly as before; `gui.py` does not replace it, it imports the two functions and builds a separate interface around them.

3 The GUI (`gui.py`)

Glossary term — Tkinter: Python's standard, built-in library for building desktop windows with buttons, text boxes, and other controls ("widgets"). It ships with the standard CPython installer on Windows and macOS; on Linux it is sometimes a separate OS package (`python3-tk`).

Glossary term — `ttk`: a themed extension of Tkinter's basic widgets (buttons, tabs, frames) that supports more modern-looking styling; `gui.py` uses it for the tab bar, buttons, and frames, while using plain Tkinter widgets for the multi-line text boxes and the canvas.

3.1 Widget tree



Both tabs are built by the same helper function, `_build_conversion_tab`, called twice with different labels and a different conversion function (`_safe_text_to_morse` or `_safe_morse_to_text`, which are thin wrappers that just call `text_to_morse`/`morse_to_text`). This is why the two tabs look and behave identically except for which direction they convert: they share one code path instead of being duplicated.

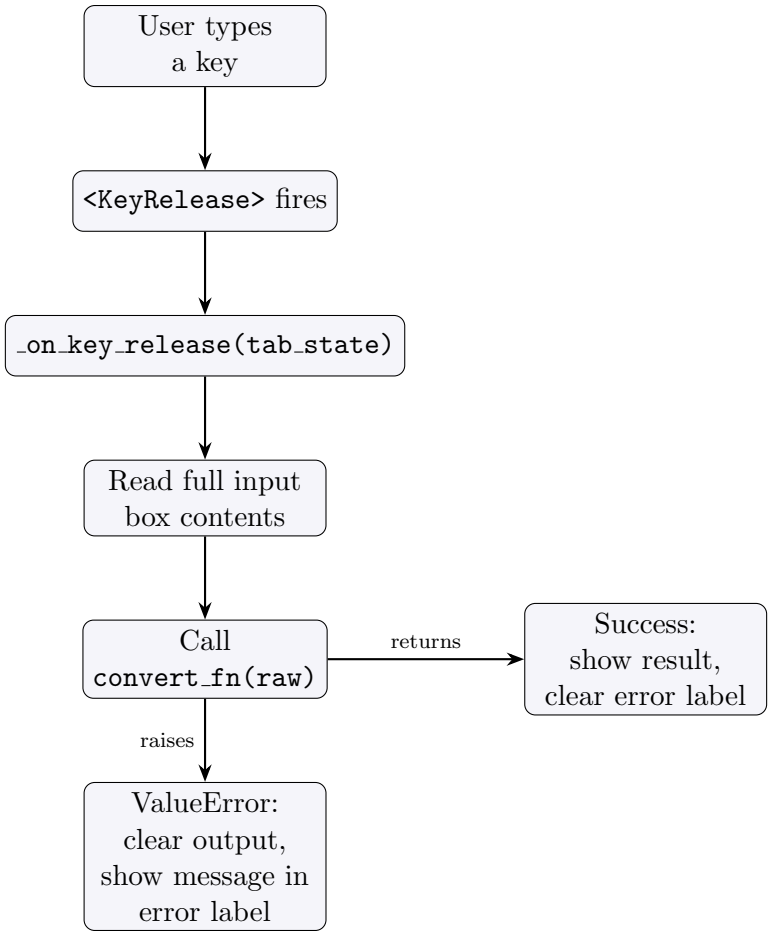
Each tab's state (its input box, output box, error label, and which conversion function it uses) is stored in a small dictionary, `tab_state`, rather than as separate named variables, so the same event-handling code can operate on "whichever tab this is" generically.

3.2 Live conversion on every keystroke

The input `Text` widget of each tab is bound to the `<KeyRelease>` event — a callback that fires every time a key is released while that widget has focus:

```
input_box.bind(
    "<KeyRelease>", lambda event, s=tab_state: self._on_key_release(s)
)
```

`lambda` here defines a small anonymous function used only as this one callback; `s=tab_state` captures the specific tab's state dictionary at binding time, so both tabs' input boxes call the same `_on_key_release` method but each with its own data.



Some implementation details worth noting:

- The output `Text` widget is created with `state="disabled"` so the user cannot type into it directly; `_on_key_release` briefly flips it back to `"normal"` to update its contents, then disables it again.
- If the input box is empty (after stripping whitespace), the handler clears the output and the error label and returns early, instead of trying to convert an empty string.
- Because the callback reruns on *every* keystroke, it also runs on invalid intermediate states — for example, a partially typed Morse sequence. This is expected: the `ValueError` is caught at this layer precisely so a partial or invalid input shows a small inline message instead of stale output or a crash.

3.3 Copy and clear

`_copy_to_clipboard` reads the output box's full text (from position `"1.0"`, meaning line 1 column 0, to `"end-1c"`, the end of the text minus the automatic trailing newline Tkinter always adds) and places it on the system clipboard using Tkinter's built-in `clipboard_clear`/`clipboard_append`, needing no third-party library. `_clear` empties both boxes and the error label for that tab.

4 The playback panel

The playback panel sits below the notebook and is shared by both tabs rather than duplicated per tab: there is one `Canvas` (a Tkinter widget for drawing simple shapes) and one `Play` button.

4.1 Choosing which Morse string to play

Because playback is shared, it needs a rule for which tab's content to animate when the user presses Play. `_current_tab_state` implements that rule:

```
for state in (self.text_to_morse_tab, self.morse_to_text_tab):
    box = (state["output_box"] if state["playback_source"] == "output"
           else state["input_box"])
    content = box.get("1.0", "end-1c").strip()
    if content:
        return content
return ""
```

Each tab was tagged at construction time with a `playback_source`: the Text-to-Morse tab is tagged `"output"` (its *output* box already holds Morse code), and the Morse-to-Text tab is tagged `"input"` (its *input* box is the one that holds Morse code, since its output box holds plain text). The function checks the Text-to-Morse tab first; if that tab's Morse output is non-empty, it wins. Only if it is empty does the function fall back to the Morse-to-Text tab's input. This is a priority order, not a "most recently edited" rule — if both tabs happen to have content, the Text-to-Morse tab's output always plays.

4.2 Rendering dots, dashes, and gaps (updated for real timing)

Playback used to walk only the dot/dash characters with one flat, approximate gap between every symbol. It has since been rebuilt around `build_timing_sequence(morse)`, a function that now lives in `morse_audio.py` (§5) because both the visual flash and the new audio tones need to read off the exact same timeline. `_start_playback` calls it once and gets back an ordered list of `(kind, units)` tuples, where `kind` is `"dot"`, `"dash"`, or `"gap"` and `units` is a small integer expressing standard Morse timing (see §5.1 for exactly how those unit counts are derived).

`_render_events` then turns that list into shapes on the canvas, left to right, converting each event's `units` into real milliseconds by multiplying by `UNIT_MS` (imported from `morse_audio.py`, 80 ms):

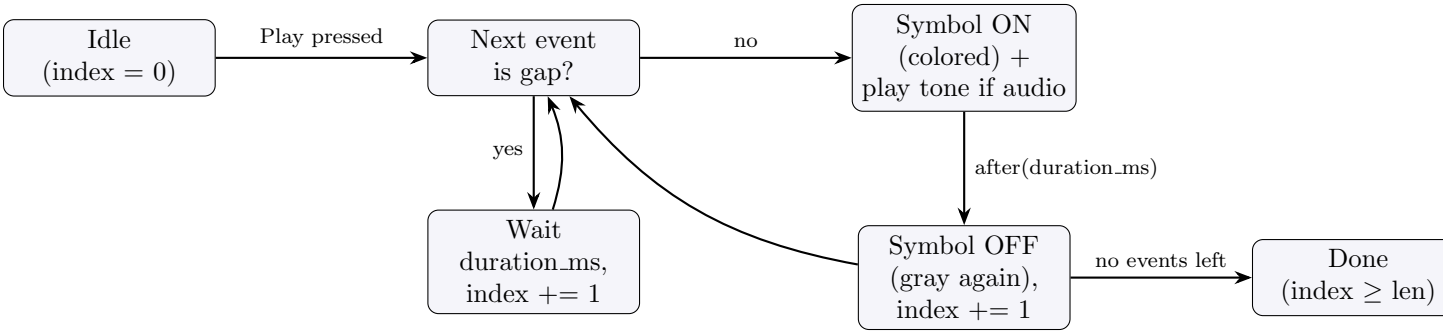
```
for kind, units in events:
    duration_ms = units * UNIT_MS
    if kind == "gap":
        prepared.append({"kind": "gap", "duration_ms": duration_ms, "box": None})
        x += 6 + units * 4
        continue
    width = 14 if kind == "dot" else 34
    box = self.playback_canvas.create_oval(...) if kind == "dot" \
        else self.playback_canvas.create_rectangle(...)
    prepared.append({"kind": kind, "duration_ms": duration_ms, "box": box})
    x += width
```

A dot gets a small oval, a dash a wider rectangle, both initially filled with `SYMBOL_OFF` (muted gray); a gap gets no shape at all (`"box": None`) but is still a real entry in the list with its own

duration, and it also widens the horizontal spacing on the canvas so the eye can see letter and word boundaries (wider gaps get more empty space, not just more time). Each prepared dictionary is stored in `self._playback_events`, the single source of truth the timing step below reads from.

4.3 Timing state machine (drives both the flash and the tone)

Playback advances through `self._playback_events` one entry at a time using `root.after(ms, callback)` — Tkinter's way of scheduling a function to run once, after a delay, without blocking the rest of the program (there is no separate thread; Tkinter's own event loop simply calls the function later). Unlike the previous fixed-millisecond version, every duration here comes from the event's own `duration_ms` (`units` × 80 ms), so a dot lights for 80 ms, a dash for 240 ms, and the different gap sizes (80 ms within a letter, 240 ms between letters, 560 ms between words) are honored exactly rather than approximated by one flat pause.



Walking the code behind this diagram, `_advance_playback` is the function that both starts each event and is scheduled to run again for the next one:

```
def _advance_playback(self) -> None:
    if self._play_index >= len(self._playback_events):
        self._playback_job = None
        return
    event = self._playback_events[self._play_index]
    if event["kind"] == "gap":
        def next_event():
            self._play_index += 1
            self._advance_playback()
        self._playback_job = self.root.after(event["duration_ms"], next_event)
        return
    box = event["box"]
    color = DOT_ON if event["kind"] == "dot" else DASH_ON
    self.playback_canvas.itemconfigure(box, fill=color)
    if self._audio_backend is not None:
        tone_path = self._dot_tone_path if event["kind"] == "dot" else self.
            _dash_tone_path
        try:
            play_wav_async(tone_path, self._audio_backend)
        except OSError:
            pass # visual flash still works even if this one playback call fails
    def turn_off():
        self.playback_canvas.itemconfigure(box, fill=SYMBOL_OFF)
        self._play_index += 1
```

```
        self._advance_playback()
    self._playback_job = self.root.after(event["duration_ms"], turn_off)
```

This is still *recursive scheduling*: `_advance_playback` does not loop internally; each call schedules exactly one future call via `root.after` (either `next_event` for a gap, or `turn_off` for a dot/-dash, which itself schedules the next `_advance_playback`), and the chain naturally ends when `_play_index` reaches the end of the event list. The audio call sits right where the shape turns on, and §5.4 explains why it has to be non-blocking for the flash and the tone to actually land at the same moment. A caught `OSError` around `play_wav_async` means a playback failure (e.g. the player process refusing to start) can never break the visual flash, which is treated as an honest, self-sufficient representation on its own.

Pressing Play again while an animation is already running cancels the in-flight schedule first (`self.root.after_cancel(self._playback_job)` in `_start_playback`) before starting a fresh one, so repeated presses cannot stack up multiple concurrent animations fighting over the same canvas shapes.

5 Audio playback (`morse_audio.py`)

This whole module is new. It has two jobs: turn Morse timing into actual sound (a WAV file full of sine-wave tones), and figure out, without assuming anything, how to play that sound on whatever machine the app happens to be running on — falling back honestly to visual-only playback if it can't.

Glossary term — WAV file: a common, simple audio file format that stores uncompressed digital sound as raw sample numbers plus a small header describing the format (channels, sample rate, bit depth). It needs no special decoder; Python's stdlib `wave` module can read and write it directly.

Glossary term — PCM (pulse-code modulation): the raw-sample representation WAV stores: sound as a plain sequence of numbers, one per tiny slice of time, each number representing the speaker's position at that instant. "16-bit PCM" means each number is a 16-bit integer.

Glossary term — sine wave: the smoothest, simplest kind of sound wave, a single pure tone with no overtones, produced mathematically by the `sin` function. This project literally computes `math.sin` at each sample to build its tone.

5.1 Morse timing, expressed in units (`build_timing_sequence`)

International Morse Code timing is conventionally expressed as ratios, not fixed millisecond values, so it can be played at any speed by scaling one base unit:

Dot	1 unit
Dash	3 units
Gap within a letter (between that letter's own dots/dashes)	1 unit
Gap between letters	3 units
Gap between words	7 units

In this project, one unit (`UNIT_MS`) is fixed at 80 milliseconds, so a dot lasts 80 ms, a dash 240 ms, and so on. `build_timing_sequence(morse)` walks a Morse string in the same shape `text_to_morse`

produces and `morse_to_text` consumes (letters space-separated, words separated by `" / "`) and emits an ordered list of `(kind, units)` tuples — exactly the list `gui.py` renders in §4.3–4.4:

```
words = [w for w in morse.strip().split(" / ") if w != ""]
for w_index, word in enumerate(words):
    letters = [letter for letter in word.split(" ") if letter != ""]
    for l_index, letter in enumerate(letters):
        symbols = [c for c in letter if c in ".-"]
        for s_index, symbol in enumerate(symbols):
            kind = "dot" if symbol == "." else "dash"
            units = DOT_UNITS if symbol == "." else DASH_UNITS
            events.append((kind, units))
            if s_index < len(symbols) - 1:
                events.append(("gap", INTRA_SYMBOL_GAP_UNITS))
        if l_index < len(letters) - 1:
            events.append(("gap", INTER_LETTER_GAP_UNITS))
    if w_index < len(words) - 1:
        events.append(("gap", INTER_WORD_GAP_UNITS))
```

The three nested loops mirror the three levels of Morse structure (words → letters → individual dot/dash symbols); a gap is only appended *between* items, which is why each loop checks it is not on the last item (`s_index < len(symbols) - 1`, etc.) before adding one — there is no trailing gap after the very last symbol of the very last letter of the very last word. Because this is the single function both the canvas (§4.3) and the audio renderer below read from, the visual flash and the tone are guaranteed to describe the same timeline; there is no separate, independently-tuned copy of the timing logic for audio versus visuals.

5.2 Generating a tone (`_sine_pcm_frames`, `write_tone_wav`)

```
def _sine_pcm_frames(duration_ms, frequency=TONE_FREQUENCY_HZ,
                     sample_rate=SAMPLE_RATE, volume=VOLUME):
    n_samples = int(sample_rate * duration_ms / 1000)
    amplitude = int(volume * 32767)
    samples = [
        int(amplitude * math.sin(2 * math.pi * frequency * (i / sample_rate)))
        for i in range(n_samples)
    ]
    return struct.pack("<%dh" % len(samples), *samples)
```

Step by step: `n_samples` is how many individual numbers are needed to cover the requested duration at the chosen sample rate (44,100 samples per second, CD quality, `SAMPLE_RATE`); `amplitude` scales a 0–1 volume fraction (`VOLUME = 0.5`, half volume) up to the largest value a 16-bit signed integer can hold (32767), since PCM samples for a 700 Hz tone (`TONE_FREQUENCY_HZ`) are 16-bit signed integers; the list comprehension computes one sample per time-step `i` by evaluating the sine function at that instant, scaled to that amplitude; **Glossary term — `struct.pack`:** a stdlib function that converts Python numbers into the exact raw bytes a file format expects. `"<%dh" % len(samples)` builds a format string meaning "this many little-endian (`<`) 16-bit signed integers (`h`)", so the whole sample list becomes one contiguous byte string ready to write to disk. A parallel function, `_silence_pcm_frames`, does the same thing but with every sample fixed at zero, used to render the gaps between tones when rendering a whole message (§5.3).

`write_tone_wav(path, duration_ms, ...)` calls `_sine_pcm_frames` once and writes the result with the stdlib `wave` module:

```
with wave.open(path, "wb") as wav_file:
    wav_file.setnchannels(1)
    wav_file.setsampwidth(2)
    wav_file.setframerate(sample_rate)
    wav_file.writeframes(frames)
```

`setnchannels(1)` means mono (one audio channel, not stereo); `setsampwidth(2)` means each sample is 2 bytes (16 bits), matching the `struct.pack` format above; `setframerate` records the same sample rate used to generate the frames, so any player reading the file knows how fast to play the samples back. `wave.open` used as a `with` block (a *context manager*, explained further in §5.3's sibling function) guarantees the file's header gets finalized and the file closed even if something goes wrong partway through.

5.3 Rendering a whole message (`render_morse_to_wav`) and pre-building tones (`build_symbol_tones`)

`render_morse_to_wav(morse, path)` is the offline counterpart to live playback: it calls `build_timing_sequence` once, then walks every event, appending either sine-wave frames (for a dot/dash) or silence frames (for a gap) of the correct duration into one growing `bytearray`, and writes the whole thing as a single WAV file. It is not used during a normal live GUI session; it exists for offline rendering of a full message to one file, and it is what made it possible to verify the timing independently of the live playback path (see §5.6).

Live playback, in contrast, never renders a whole message to one file: it renders exactly two short WAV files once, at startup, and reuses them for every dot and every dash for the rest of the session. That is `build_symbol_tones()`'s job:

```
def build_symbol_tones(unit_ms=UNIT_MS):
    tone_dir = _tone_dir()
    dot_path = os.path.join(tone_dir, "dot.wav")
    dash_path = os.path.join(tone_dir, "dash.wav")
    write_tone_wav(dot_path, DOT_UNITS * unit_ms)
    write_tone_wav(dash_path, DASH_UNITS * unit_ms)
    return dot_path, dash_path
```

`_tone_dir()` lazily creates a fresh temporary directory once per process with `tempfile.mkdtemp` (a stdlib call that makes a uniquely-named directory meant for throwaway files) and registers its deletion with `atexit.register(shutil.rmtree, ...)` — a stdlib hook that runs a chosen function automatically when the Python process is about to exit normally, so the two small WAV files this app creates are cleaned up from the OS's temp storage without the user ever noticing they existed. Generating the tones once and reusing the files (rather than writing a fresh WAV on every keystroke's Play press, or per symbol during playback) avoids repeated disk writes and sine-wave computation for something that never changes: a dot always sounds like a dot.

5.4 Choosing a playback backend (`detect_playback_backend`)

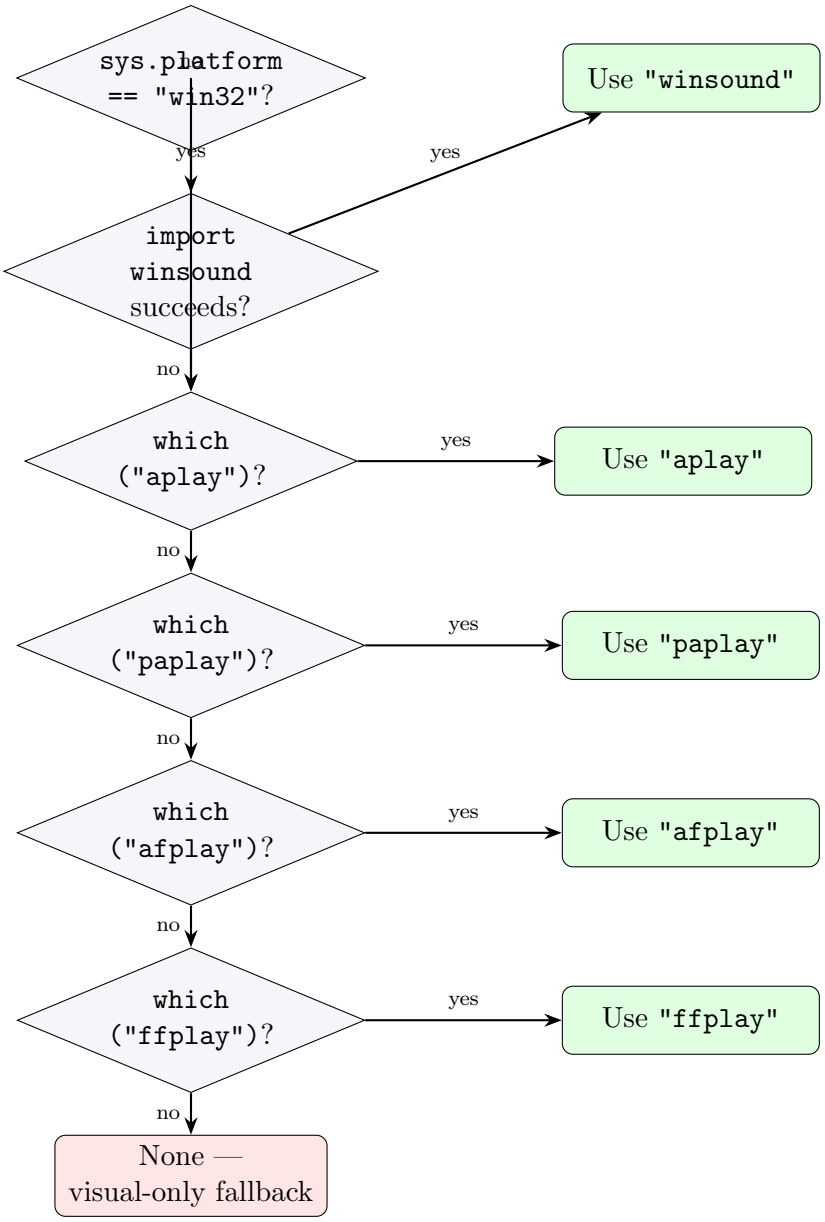
Having a WAV file is only half the problem: something has to actually play it through the speakers, and Python's standard library has no built-in, cross-platform way to do that. `winsound` (Windows-

only, stdlib) can play a WAV file directly, but nothing built into stdlib does the equivalent on Linux or macOS. This project's answer is to check, at runtime, what the current machine actually has — never to assume a platform's "obvious" tool is present — and use the first working option:

```
def detect_playback_backend():
    if sys.platform == "win32":
        try:
            import winsound
            return "winsound"
        except ImportError:
            pass
    for player in _PLAYERS_IN_PRIORITY_ORDER:
        if shutil.which(player):
            return player
    return None
```

Glossary term — `shutil.which`: a stdlib function that searches the system's `PATH` (the list of directories the OS checks for runnable programs) for a command by name, returning its location if found or `None` if it isn't installed — the same mechanism a shell uses to resolve a command you type. This is how the function checks for a command-line player without ever importing or running it speculatively.

`_PLAYERS_IN_PRIORITY_ORDER` is the tuple `("aplay", "paplay", "afplay", "ffplay")`. The check order matters: on Windows, `winsound` always wins if importable (it is the most direct, dependency-free option there); everywhere else, the four command-line players are tried in that fixed order, and the first one actually found on the machine's `PATH` is the one used. If none of the five checks succeed, the function returns `None`, and §5.7 covers exactly what the app does with that.



5.5 Fire-and-forget playback (`play_wav_async`) and why non-blocking matters

```
def play_wav_async(path, backend):
    if backend == "winsound":
        import winsound
        winsound.PlaySound(path, winsound.SND_FILENAME | winsound.SND_ASYNC)
        return None
    if backend == "aplay":
        command = ["aplay", "-q", path]
    elif backend == "paplay":
        command = ["paplay", path]
    elif backend == "afplay":
        command = ["afplay", path]
    elif backend == "ffplay":
        command = ["ffplay", "-nodisp", "-autoexit", "-loglevel", "quiet", path]
```

```
    else:
        raise ValueError(f"Unknown playback backend: {backend!r}")
    return subprocess.Popen(command, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
        )
```

Glossary term — `subprocess`: the stdlib module for launching another program (an external command) from Python and optionally waiting for it, reading its output, etc. `subprocess.Popen` starts the external program and returns immediately, handing back a handle to the still-running process; this is different from `subprocess.run` (stdlib's other common entry point), which *waits* for the program to finish before returning. This function deliberately uses `Popen`, not `run`, which is why the docstring and this section both call it "fire-and-forget": the command-line player is launched and Python moves on immediately, without knowing or caring when the tone actually finishes playing. On Windows, `winsound.PlaySound` is given the `SND_ASYNC` flag, which is the equivalent non-blocking request for that API: play this sound and return immediately, do not wait.

This non-blocking behavior is not a minor implementation detail, it is the reason the visual flash and the audio tone stay together at all. `gui.py`'s `_advance_playback` (§4.4) is driven entirely by Tkinter's own `root.after` timer, on a schedule computed from `build_timing_sequence`'s unit list. If `play_wav_async` blocked until the external player process actually finished making sound — which is exactly what would happen if it called `subprocess.run(command)` or `Popen(...).wait()` instead — then every dot and dash would pause the whole GUI (including its own timer) for however long that player took to exit, and the canvas flash would visibly lag behind, or race ahead of, the sound. Because `Popen` (and `SND_ASYNC`) return control immediately, Tkinter's timer keeps running the visual side on its own precise schedule while the OS plays the tone independently in the background; both sides are reading off the exact same `build_timing_sequence` event list (§5.1), so they start each symbol at the same moment even though neither one waits for the other to finish.

5.6 The honest fallback

If `detect_playback_backend()` returns `None` (§5.5, bottom of the flowchart), `gui.py` never tries to build tone files or call `play_wav_async` at all — it just sets `self._audio_backend = None` and skips straight to a status label reading exactly:

> ``No audio output available on this system, showing visual playback only.''

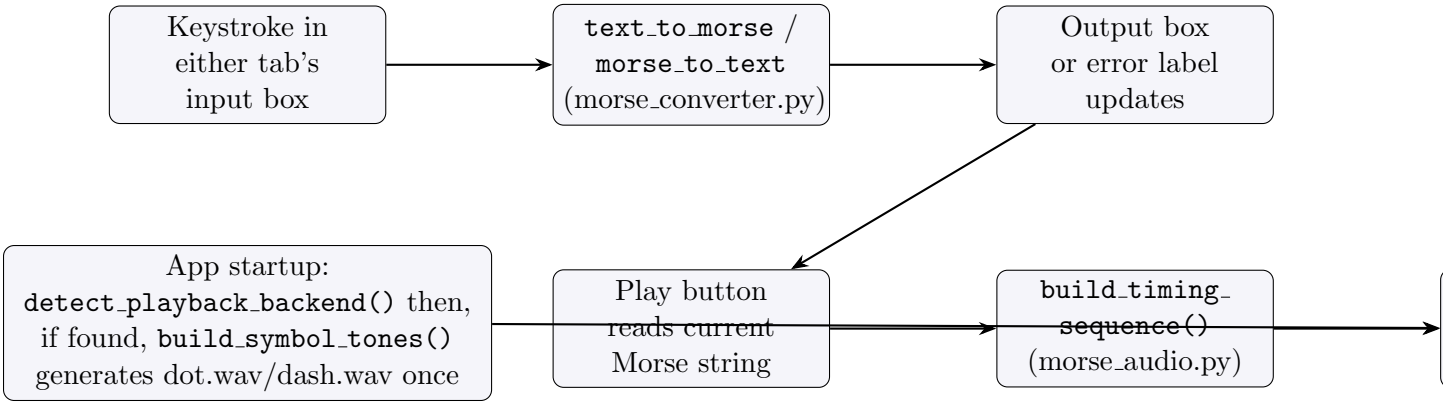
The visual flash keeps working exactly as before; nothing crashes, and nothing silently no-ops without telling the user why. This mirrors a design choice already present in the conversion logic (§2.2–2.3): prefer a clear, specific message over letting an error escape unexplained or pretending everything succeeded.

5.7 What was actually verified

Per the project's honesty standard, this section separates what was run and checked from what is merely implemented. In the Linux container this project was built and verified in: both `aplay` and `ffplay` were present on the `PATH` (`paplay` and `afplay` were not); `detect_playback_backend()` picked `"aplay"` (earlier in the priority order), and playing a rendered WAV through it was confirmed to actually work — the `aplay` subprocess exited with code 0. A WAV rendered for the message "SOS" was confirmed to be a valid RIFF/WAVE file (checked with the `file` command) and its duration matched the expected duration computed independently from the timing units exactly: 2160 ms expected versus 2160.0 ms actual.

The `winsound` (Windows) and `afplay` (macOS) code paths are implemented and use documented, stable APIs, but were *not* run on Windows or macOS in this project — only the Linux/`aplay` path was exercised end to end. This distinction is stated explicitly rather than implied, matching the README's own wording on the same point.

6 Summary of data flow



The bottom box happens once, at app startup, before any typing or playback: the app decides whether audio is possible at all and, if so, pre-generates the two tone files that `play_wav_async` reuses on every Play press thereafter (§5.4). Everything above it happens live, per keystroke or per Play press.

7 What is intentionally out of scope

Per the README, there is no persistence (nothing is saved to disk), no network calls, and no configuration file, and no automated unit tests committed to the repository. Playback is no longer visual-only: it now plays real audio tones too (§5), generated entirely from the standard library. What remains genuinely out of scope on the audio side: there is no real cross-platform audio device API in use (the app shells out to whatever command-line player the OS already has, rather than writing samples directly to a sound device), so there is a small, unavoidable process-spawn delay per symbol and no way to confirm a given tone actually finished playing — playback is fire-and-forget, timed against the visual clock rather than the player subprocess's own lifecycle (§5.4). A real cross-platform audio library would remove that gap but would pull in a third-party dependency this project deliberately avoids.